LO6 — Container Orchestration: Theoretical Questions & Answers

1. Analyze the networking model of Kubernetes versus Docker's network.

Kubernetes uses a flat, cluster-wide network model where every Pod gets its own IP and can communicate with any other Pod without NAT, enforced by CNI plugins (Calico, Flannel, Cilium). Docker's default bridge network isolates containers on a single host, requiring explicit port mapping or user-defined networks for inter-container communication. Kubernetes therefore natively supports distributed, multi-host networking, while Docker's model is simpler but host-scoped by default.

2. Evaluate Kubernetes storage abstraction (CSI, StorageClasses, dynamic provisioning) against Podman's storage plugins. Which is more flexible for stateful workloads, and why?

Kubernetes CSI with StorageClasses and dynamic provisioning allows on-demand volume creation across cloud and on-premise backends, decoupled from the workload definition, making it far more flexible for stateful workloads at scale. Podman's storage relies on local volume plugins and lacks native dynamic provisioning or a unified lifecycle for distributed storage, limiting it mainly to single-host use cases. Kubernetes wins decisively for stateful workloads requiring portability, replication, and automated storage lifecycle management.

3. Evaluate the operator pattern (CRDs + controllers) for managing stateful services versus using only k8s manifests, in terms of control and effort.

The operator pattern encodes domain-specific operational knowledge (backups, failover, schema migrations) into a controller, giving fine-grained automated lifecycle control that plain manifests cannot express. Raw Kubernetes manifests require manual intervention for day-2 operations like scaling a database cluster or performing rolling upgrades safely, increasing operational effort significantly. Operators demand more upfront development effort but vastly reduce ongoing operational burden for complex stateful services.

4. Assess the developer experience of plain Kubernetes (kubectl + manifests) versus OpenShift's Source-to-Image (S2I) and developer console. What does each optimize for?

Plain Kubernetes with kubectl optimizes for flexibility and control, exposing the full API surface but requiring developers to understand YAML manifests, networking, and RBAC from the start. OpenShift's S2I abstracts the container build process — developers push source code and the platform builds and deploys it — optimizing for speed-to-deployment and a lower barrier to entry. OpenShift is developer-experience-first; plain Kubernetes is operator/platform-engineer-first.

5. Critically evaluate migrating a workload from OpenShift to Kubernetes.

Migration is feasible since OpenShift is built on Kubernetes, but workloads relying on OpenShift-specific APIs (Routes, BuildConfigs, DeploymentConfigs, S2I) require refactoring to Kubernetes-native equivalents (Ingress, Dockerfile-based builds, Deployments). OpenShift's stricter default security context constraints (SCCs) mean workloads may actually run with fewer restrictions on vanilla Kubernetes, which could introduce undetected privilege escalation risks. Teams must also replace integrated OpenShift tooling (image registry, monitoring, CI/CD pipelines) with self-managed alternatives, substantially increasing operational overhead.

6. Evaluate running CI/CD build agents (Jenkins agents, GitLab runners, Tekton) inside Kubernetes versus on dedicated VMs, considering isolation, autoscaling, and noisy-neighbor effects.

Running build agents in Kubernetes enables elastic autoscaling (agents spin up per job and terminate after), reducing idle costs and improving utilization, with Tekton being natively designed for this model. Isolation is weaker than dedicated VMs — shared node resources mean a resource-hungry build job can create noisy-neighbor effects unless strict resource limits and node affinity/taints are used. Dedicated VMs offer stronger isolation and predictable performance but waste resources when idle and require manual capacity management.

7. How does Kubernetes integrate with cloud environments and dynamic capacity provisioning?

Kubernetes integrates deeply with cloud providers via the Cloud Controller Manager, enabling automatic provisioning of cloud load balancers, persistent volumes, and cloud-specific node groups. The Cluster Autoscaler watches pending pods and dynamically provisions or decommissions nodes in cloud node pools, aligning compute capacity with actual workload demand in real time. This tight integration makes Kubernetes the de facto standard for cloud-native applications requiring elastic, pay-for-use infrastructure.

8. Critically evaluate the default security and hardening of OpenShift versus vanilla Kubernetes. Why do enterprises in regulated industries often choose OpenShift?

Vanilla Kubernetes ships with permissive defaults — no mandatory pod security, optional audit logging, and no built-in image scanning — requiring significant hardening effort to meet compliance requirements. OpenShift enforces Security Context Constraints by default, ships with integrated image vulnerability scanning (via Quay integration), mandatory audit logging, and FIPS-compliant builds, delivering a hardened baseline out of the box. Regulated industries (finance, healthcare, government) choose OpenShift because it reduces compliance gap closure effort and comes with Red Hat's enterprise support and certification against standards like FIPS 140-2.

9. Critically evaluate the learning curve and required skill set of OpenShift versus plain Kubernetes. Does OpenShift's added abstraction help or hinder a team new to containers?

Plain Kubernetes has a steep learning curve requiring understanding of the full API, YAML manifests, networking primitives, and RBAC; OpenShift adds its own concepts (Projects, Routes, SCCs, OperatorHub) on top of this, creating a doubly complex initial surface area. For teams genuinely new to containers, OpenShift's developer console and S2I can accelerate early wins by hiding infrastructure complexity, so abstraction helps at the application layer but hinders at the platform-administration layer. The net verdict is that OpenShift helps developers but demands more specialized skill from the platform team responsible for installation, upgrades, and operations.

10. Compare the resource footprint of full Kubernetes versus k3s for a small on-premise deployment. When is full Kubernetes overkill?

A standard Kubernetes control plane (etcd, API server, scheduler, controller-manager, cloud-controller) requires roughly 2–4 GB of RAM and multiple nodes just for HA, whereas k3s bundles all control plane components into a single ~60 MB binary and runs comfortably in under 512 MB. Full Kubernetes is overkill when the deployment targets edge devices, IoT gateways, development environments, or small teams with fewer than a handful of services that do not require multi-zone HA. k3s delivers the same Kubernetes API with a fraction of

the resource cost, making it the pragmatic choice for resource-constrained on-premise scenarios.

11. Defend whether a small team should use Docker Compose on a single host or move straight to Kubernetes, considering complexity, scaling needs, and resilience.

For a small team running a handful of services without strong scaling or HA requirements, Docker Compose on a single host is the correct choice — it is declarative, simple to operate, and has near-zero operational overhead compared to managing a Kubernetes cluster. Kubernetes introduces significant complexity (networking, RBAC, storage, control plane management) that consumes engineering time a small team cannot afford to spend on infrastructure instead of product. Moving to Kubernetes is justified only when the team genuinely needs horizontal scaling, multi-host resilience, or rolling deployments — not as a premature architectural investment.

12. Analyze vendor lock-in across Kubernetes (open source / CNCF) and OpenShift (Red Hat). How does the choice affect long-term flexibility?

Kubernetes itself is CNCF-governed open source, and workloads using only upstream APIs are portable across any conformant distribution (EKS, GKE, AKS, k3s), minimizing long-term lock-in at the platform level. OpenShift uses proprietary APIs (Routes, BuildConfigs, DeploymentConfigs, SCCs) and is commercially supported exclusively by Red Hat, creating meaningful migration friction if the team later wants to switch distributions or cloud providers. The long-term strategic risk of OpenShift lock-in is manageable if teams discipline themselves to use only upstream Kubernetes APIs and treat OpenShift features as optional conveniences, but in practice adoption of OpenShift-specific tooling tends to deepen over time.

13. Defend a recommendation of OpenShift over vanilla Kubernetes for a company that wants an integrated, vendor-supported platform with built-in developer and operations tooling.

OpenShift delivers a fully integrated platform — bundling a container registry, CI/CD pipelines (Tekton/ArgoCD via OperatorHub), monitoring (Prometheus/Grafana), log aggregation, and a developer console — that would each require separate procurement, integration, and ongoing maintenance if assembled from scratch on vanilla Kubernetes. Red Hat's enterprise subscription provides SLA-backed support, certified operators, and a clear upgrade path, which eliminates the staffing overhead of maintaining upstream component compatibility yourself. For a company that wants to move fast without building a platform engineering team from scratch, OpenShift's total cost of ownership is often lower despite the higher licensing cost.

14. Compare storage provisioning between Kubernetes and regular VM workloads.

VM workloads typically provision storage manually or via infrastructure automation (Terraform, Ansible) with direct block/NFS/SAN attachment, requiring coordination between storage, networking, and server teams and offering little self-service capability. Kubernetes, through CSI drivers and StorageClasses, enables developers to self-service persistent volumes via PersistentVolumeClaims with dynamic provisioning — storage is created, attached, and deleted automatically as workloads are scheduled. This shifts storage from an infrastructure team bottleneck to a developer-controlled resource, dramatically accelerating deployment velocity for stateful applications.

15. A startup with two engineers must ship a containerized product quickly and cheaply. Recommend a container solution and justify your choice on cost and operational overhead.

Docker Compose deployed on a single managed VM (or a small PaaS like Railway, Render, or Fly.io) is the right choice — it requires no cluster management, has a trivial learning curve, and can be operational in hours rather than weeks. A managed PaaS layering containers further reduces overhead by handling TLS, deployments, and basic scaling without any infrastructure expertise, keeping both engineers focused on product code. Kubernetes at this stage is an anti-pattern: its operational complexity would consume a disproportionate fraction of a two-person team's total engineering capacity.

16. Compare Docker and Podman in terms of architecture (daemon vs daemonless) and rootless security. Why might a security-conscious team prefer Podman?

Docker uses a long-running root daemon (dockerd) as the central intermediary for all container operations, meaning any vulnerability in the daemon is a root-level privilege escalation on the host. Podman is daemonless — each container is a direct child process of the invoking user — and supports fully rootless operation, so containers run under the invoking user's UID with no elevated host privileges. Security-conscious teams prefer Podman because eliminating the root daemon removes a critical attack surface, and rootless containers enforce the principle of least privilege by default without requiring additional configuration.

17. Compare the day-2 operational overhead (cluster upgrades, scaling nodes, backups) of self-managed Kubernetes versus a managed Kubernetes service.

Self-managed Kubernetes requires the operations team to manually orchestrate control plane upgrades (etcd snapshots, API server version alignment, CNI/CSI plugin compatibility), manage node pool scaling, and implement etcd backup schedules — each carrying significant risk and expertise requirements. Managed services (EKS, GKE, AKS) automate control plane upgrades, provide managed node group autoscaling, and handle etcd backups transparently, reducing cluster-level operational burden to primarily application-layer concerns. The managed-service premium is almost always justified unless the organization has regulatory requirements for full infrastructure sovereignty or existing platform engineering capacity that would otherwise sit idle.

18. A media-streaming company expects spiky, unpredictable global traffic. Recommend a containerized solution. Elaborate your choice.

A managed Kubernetes service (GKE, EKS, or AKS) with Cluster Autoscaler, Horizontal Pod Autoscaler, and a global CDN is the correct solution — it can scale streaming pods from tens to thousands within minutes in response to traffic spikes, and the managed control plane eliminates cluster-management distraction during incidents. Deploying across multiple cloud regions with global load balancing (e.g., AWS Global Accelerator or GCP Cloud CDN) addresses the geographic distribution requirement while Kubernetes' health checks and rolling deployments ensure zero-downtime releases. This combination provides the elasticity, geographic reach, and operational resilience that a media-streaming workload with unpredictable global demand demands.

19. Defend whether a company that has outgrown Docker Swarm (or a single Compose host) should migrate to Kubernetes — weigh the migration effort against the long-term benefits.

Migration from Swarm or Compose to Kubernetes is a significant one-time investment — rewriting service definitions as Helm charts or Kustomize manifests, re-architecting

networking, and training the team — but this effort pays off quickly once workloads require fine-grained autoscaling, advanced deployment strategies (canary, blue-green), or multi-tenant RBAC. Staying on Swarm means accepting architectural ceilings: Swarm has minimal active development, no advanced autoscaling, and a shrinking ecosystem, meaning the migration cost only grows over time. For any team already hitting Swarm's limits, migrating to Kubernetes is the strategically correct decision despite the short-term pain.

20. Would you choose Kubernetes as a solution for a microservices architecture? Focus on its orchestration, resilience and ecosystem.

Yes — Kubernetes is purpose-built for microservices: it orchestrates independent deployable units with health checks, rolling updates, and automatic restarts, directly matching the operational model microservices require. Its resilience primitives (liveness/readiness probes, PodDisruptionBudgets, ReplicaSets) ensure services self-heal and maintain availability contracts without manual intervention. The CNCF ecosystem (Istio/Linkerd for service mesh, Prometheus for observability, Argo for GitOps) provides the surrounding infrastructure that microservices architectures depend on, making Kubernetes the ecosystem anchor for this architectural style.

21. Would you choose Kubernetes for enterprise-grade applications? Focus on its scalability, community support and feature richness.

Yes — Kubernetes is the industry-standard platform for enterprise applications: it scales horizontally from tens to tens of thousands of pods across multi-region clusters with proven production deployments at Google, Netflix, and Airbnb scale. The CNCF community is the largest open-source infrastructure community in existence, ensuring rapid bug fixes, security patches, and feature development that no proprietary alternative can match. Its feature richness — RBAC, network policies, admission webhooks, custom controllers, storage orchestration, and rich API extensibility — provides the control surface enterprises need to meet compliance, multitenancy, and operational requirements.

22. Would you choose OpenShift as a solution for a microservices architecture? Focus on its orchestration, resilience and ecosystem.

Yes — OpenShift inherits all Kubernetes orchestration and resilience primitives while adding integrated service mesh (via OpenShift Service Mesh, based on Istio) and built-in CI/CD (Tekton pipelines), which are both essential for managing many microservices without assembling a toolchain from scratch. Its OperatorHub provides certified, automatically-updated operators for common backing services (databases, message brokers), reducing the integration work that microservices architectures require. The trade-off is Red Hat lock-in on platform-level tooling, but for teams that want a production-ready microservices platform on day one rather than month six, OpenShift accelerates time-to-value.

23. Would you choose OpenShift for enterprise-grade applications? Focus on its scalability, community support and feature richness.

Yes — OpenShift's enterprise value proposition is strongest at scale: Red Hat provides a single vendor SLA covering the entire stack (OS, container runtime, Kubernetes, monitoring, logging, registry), eliminating the multi-vendor finger-pointing that plagues self-assembled stacks in enterprise incidents. Its feature set — enterprise LDAP integration, FIPS-compliant builds, multi-cluster management (Advanced Cluster Management), and compliance operator — directly addresses enterprise governance requirements that vanilla Kubernetes requires additional tooling to satisfy. The active Red Hat and broader Kubernetes/CNCF community ensures long-term viability, and OpenShift's certified operator catalog gives enterprises confidence in third-party integrations that are tested against the specific platform version they are running.